

Operating Systems

Lecture 5

Overview

- The History of Operating Systems
- Operating System Architecture
- Coordinating the Machine's Activities
- Handling Competition Among Processes
- Security

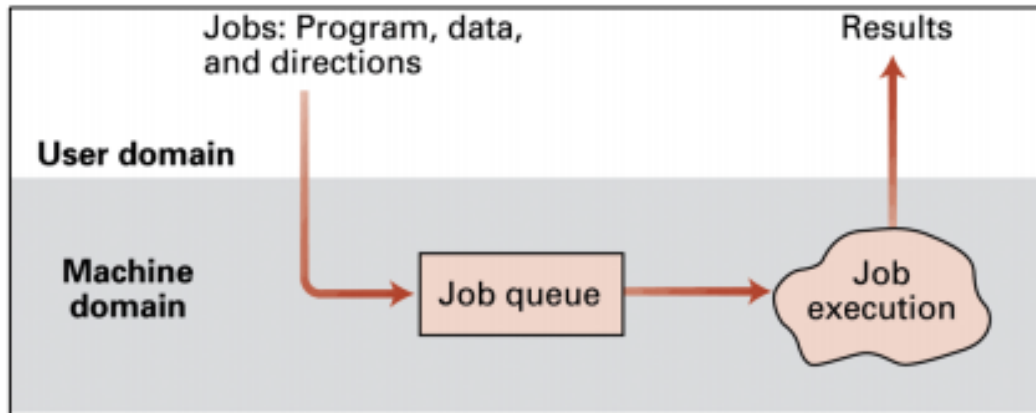
History of Operating Systems

❑ No OS (Pre 1950's):

- Computer runs a program under direct operator control

❑ Batch processing systems (1950's – 1960's):

- The OS put user programs into a job queue in a first-in-first-out manner (FIFO)
- The OS select one job at a time to run under the computer until it finishes; then it selects the next job to run



History of Operating Systems

- Simple multi-tasking systems (1960's – 1970's)
 - Several programs are arranged in the memory at the same time by the OS
 - The OS runs one program until it has to do I/O, then the OS selects another program to run
 - When the first program finishes its I/O task, it will wait for the 2nd program to finish or to pause for I/O operation.

History of Operating Systems

- Time-sharing systems (1970's – present)
 - The OS divide the CPU computation time into intervals
 - Each program entered in the job queue run for one time interval at a time
 - At the end of the time interval, the program will be paused and another program will be selected for execution.
 - The main advantage of such system is that it enables real-time interaction with multiple users at the same time.

Types of Software

- **Application software**
 - Performs specific tasks for users
 - For ex, Word is an application for word processing, Excel is an electronic spreadsheet, etc
- **System software**
 - Provides a platform for running application software
 - Consists of operating system and utility software
 - **Utility software** is a functionality which is not natively supported by an OS, but installed as a third-party add-on to the system.
 - Examples of utility software can be CD burning software, archiving, file managers, media players, etc.
- **Embedded software**
 - Such software runs on microcomputers installed inside special devices such as cars, home appliances, medical equipment, etc
 - Normally, this type of software requires minimal human interaction to operate, consumes less battery power and performs quickly/reliably.

Operating System Components

- **Main Functions of an OS:**

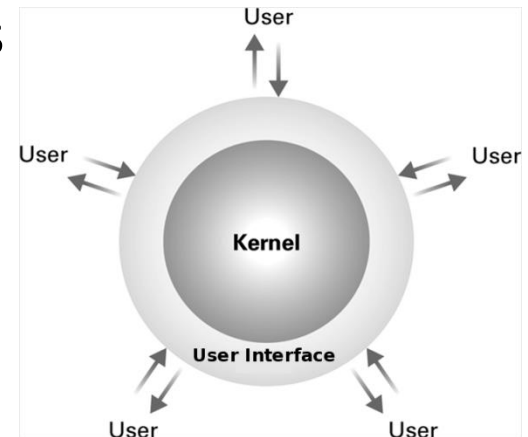
- Schedule programs for execution
- Manage main memories
- Store and retrieve files
- Provide interfaces of I/O devices to programs

- **User Interface: - Shell**

- Command Line Interface (CLI): text-based interface to interact with OS or with any app. For ex, DOS prompt, Linux bash
- Graphical user interface (GUI): user can interact with apps and the OS through windows, icons, menus and pointers. For ex, Windows Manager, Gnome.

- **Kernel: Performs basic required functions**

- File manager
- Memory manager
- Device drivers
- Scheduler and dispatcher (process manager)

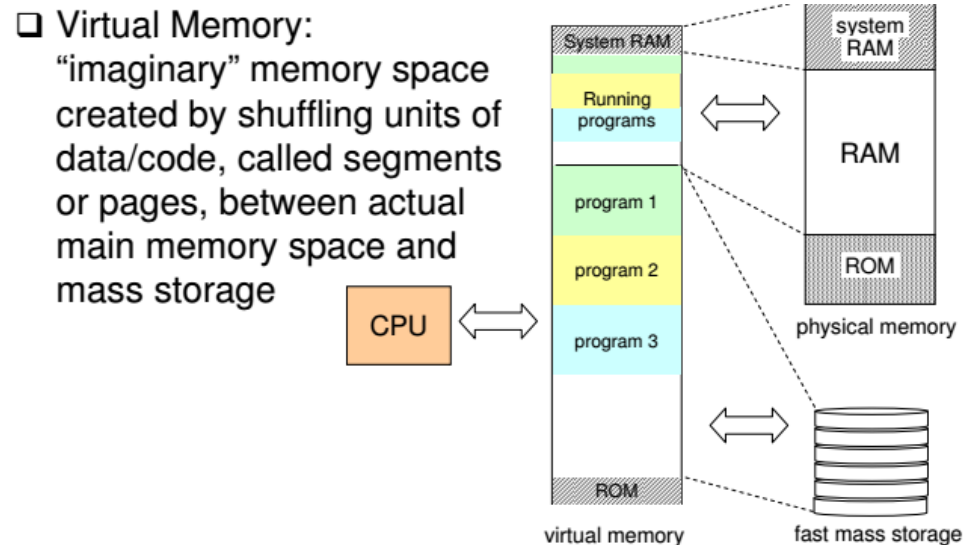


File System

- **File system** is the method and data structures that an OS uses to keep track of files on a disk/partition, that is, the way the files are organized on the disk.
 - Space management: size of sector, fragmentation/defragmentation
 - Filenames, directories and paths
 - Metadata: size, create date, owner, access permissions
 - Restricting/permitting access
- File systems vary based on the medium:
 - Disk file systems:
 - Windows: FAT (File Allocation Table), NTFS (New Technology File System)
 - Linux: ext* (ext2, ext3, ext4), XFS,
- **File manager** is key part of OS kernel which organizes the use of mass storage by maintaining the records of all files, their physical locations, user permissions, other meta-data, and available free disk space.
- Utility programs that assist working with OS's file manager:
 - Windows: Windows Explorer, Total Commander
 - Linux: Gnome commander; MacOS: Finder

Memory Manager

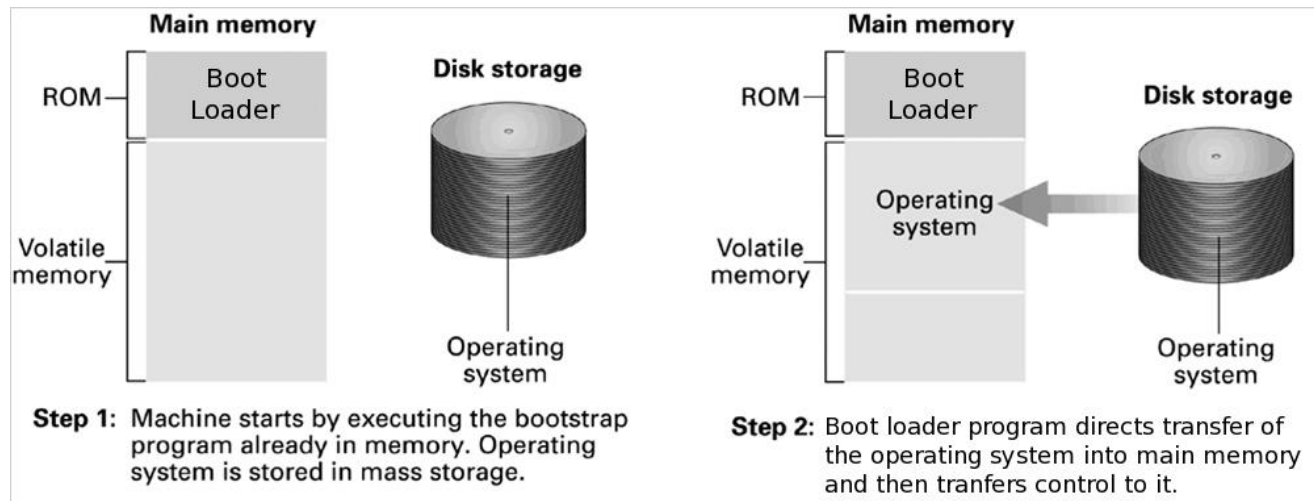
- Coordinates the memory used by multiple programs running in parallel, and restricts them from accessing the memory outside their own allotted space.
- Provides access to an abstract "virtual memory" which can be physically mapped to different parts of RAM and hard-disk.



27/30

Getting it Started (Bootstrapping)

- **Boot loader:** Program in ROM (example of firmware)
 - Run by the CPU when power is turned on
 - Transfers operating system from mass storage to main memory
 - Executes jump to operating system
- **Common firmwares:** Basic I/O System (BIOS), Extensible Firmware Interface (EFI), Open Firmware, etc



Bootstrapping Steps

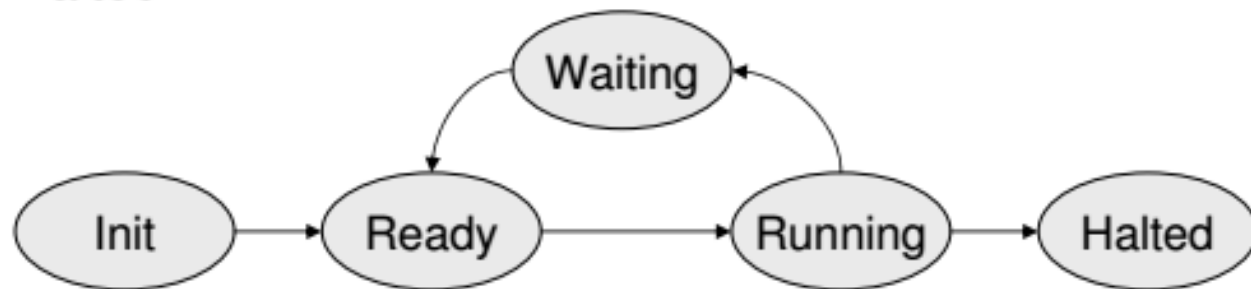
1. Power button is pressed, electrical signal goes to CPU and resets all registers, including program counter (it is set to F000). This is the address in Read-Only Memory (ROM) where special firmware (for ex, BIOS) stored.
2. CPU starts loading instructions of firmware program from ROM. First part of the firmware is Power-On Self-Test (POST) program which sends test signals to following devices in respective order:
 1. display adapter and graphic cards
 2. RAM chips
 3. keyboard, mouse
 4. CD/DVD drives
 5. Hard drive
3. Results of POST tests are compared with data stored in CMOS (battery powered memory which stores computer's device configurations)
4. BIOS searches drives in a predefined order for a **boot record** and copies it to a specified location in RAM
 - Boot record is located at track 0, head 0 and sector 1 (001) on all formatted disks and contains a **boot-up program** (512 bytes). Exact location of boot record is known for all types of mass storage.
5. BIOS jumps to an address in RAM where boot-up program is loaded
6. Boot-up program will pull the rest of the OS specific programs into memory and hands over the CPU control to OS.

Processes

- **Program:** a static set of instructions
- **Process (aka task or job):** The activity of executing a program under the control of the OS
 - Process may occupy more memory space than a program
- **Process State:** Current status of the activity
 - Program counter
 - General purpose registers
 - Related portion of main memory

Process Execution States

- ❑ When a user select a program to run in a computer, the program becomes a process and it will go through the following execution states
 - Initialization
 - Ready
 - Running
 - Waiting/Sleeping/Blocked/Interrupted
 - Halted



Scheduler

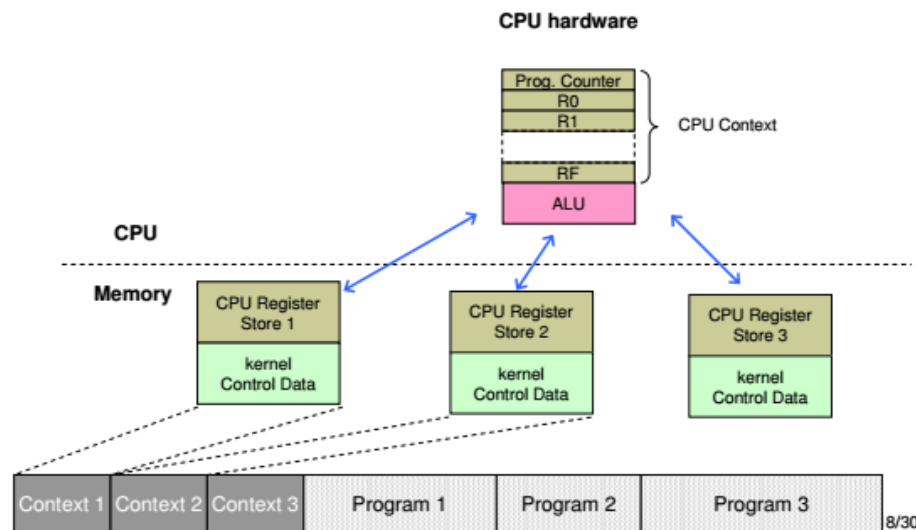
- The scheduler maintains a ***process table*** with the OS; When a user runs a program, it creates a new entry in the process table
- After the initialization, the scheduler will add it to the *ready pool* of processes
- If a process cannot continue execution for some reason (e.g. waiting for data), it will be added to the *waiting pool*
- If process is finished, the scheduler will remove it from the process table
- Scheduler can also prioritize certain processes over others

Dispatcher

- Dispatcher is the kernel component that assigns the CPU to execute a ready process
- It organizes the execution of processes into CPU ***time slices*** (usually measured in milliseconds).
- Dispatcher awards a process with a time slice through issuing an ***interrupt signal*** to CPU
 - When interrupt is received by CPU, it stops computing the current process, backs up its current state, hands-over the control to **interrupt handler** which is part of a dispatcher.
 - Interrupts can also be prioritized. For example, mouse/keyboard activity may have higher priority over most of the apps, as the computer needs to immediately react on such events
 - The highest priority interrupt is a powercut, the CPU has to backup its current state before loss of voltage.
- Dispatcher assigns CPU resources to processes that are chosen by the scheduler.

Process Switching (Context Switching)

- The operation of assigning CPU to run another task is called context switching. Context switching steps:
 - Save current states of CPU's program counter and other general-purpose registers
 - Load new task registers
- Some modern CPUs designed for multitasking already include context switching commands at the level of a machine language.

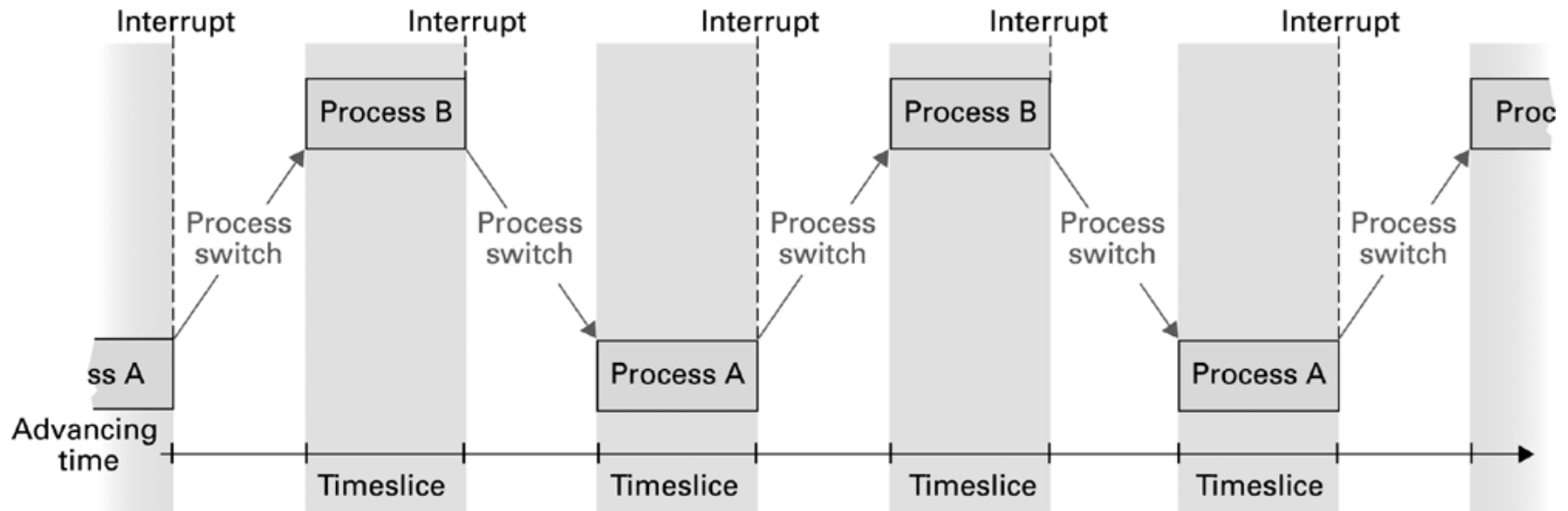


Interrupt Controller

- Interrupt Controller is a chip inside CPU which enables process switching when events such as keystrokes, mouse click, data coming from serial/parallel ports or some other software events occur
- OS maintains an *Interrupt Table* with interrupt *vectors* to find memory address to which it needs to jump in case if certain interrupt occurs.
- Once Interrupt Controller receives Interrupt Request (IRQ), CPU will make a jump to corresponding memory address in Interrupt Table where the interrupt handling routine is located
- If interrupt handler finishes successfully, it send Interrupt Return (IRET) to Interrupt Controller which will roll back the previously running process into CPU

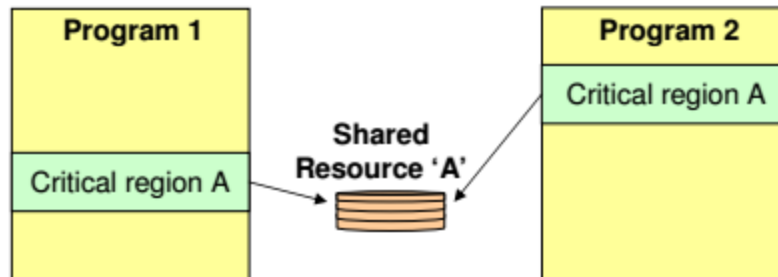
Time-sharing

- Time-sharing is one type of allocating CPU time to each process
- Each process gets the CPU for a short period of time (milliseconds) to execute.



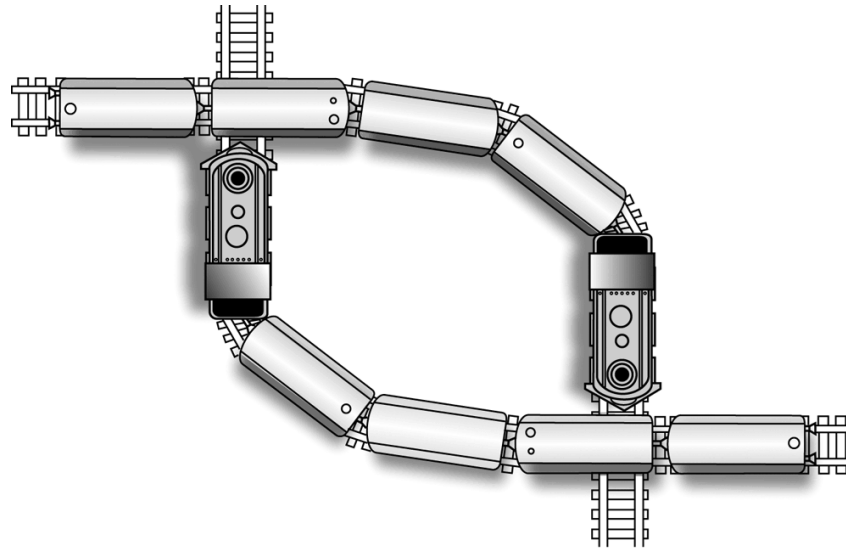
Handling Competition for Resources

- **Semaphore:** A “variable” is used to tell if a resource is in use
- **Critical Region:** Critical region is a set of instructions that must be executed by a single process
- **Mutual exclusion:** Requirement for proper implementation of a critical region



Deadlock

- Two processes block each other from continuing
- Conditions that lead to deadlock
 1. Competition for non-sharable resources
 2. At least two resource are needed by both processes
 3. An allocated resource can not be forcibly retrieved



Spooling

- OS can make non-sharable resource look as if it is sharable.
- For ex, 2 processes request printer.
- OS can simultaneously accept instructions from both processes and store them in memory.
- The processes, however, think that they are communicating with the printer.
- But, OS manages the execution of these instructions in a sequential manner.
- This technique of holding data for output at a later but more convenient time is called ***spooling***

Security – External Threats

- Most compute systems today are protected by using a login system
- Problems with a login system
 - Insecure passwords:
 - password cracker can pick the password from huge dictionary of commonly used passwords/or simply use **bruteforcing**.
 - network sniffer can capture the communication between client/server, and decode it
 - Trojan Horse, special type of malware (malicious software) which creates a backdoor to the hacker to take over infected computer
 - Sniffing software:
 - **Web sniffers**: Fiddler2, Burp suite
 - **Network sniffers**: Wireshark, Capsa, Tcpdump
- Counter measures
 - Auditing software – a program that monitors the operation of a computer and report anything abnormal to the administrator
 - Antiviruses, firewalls, spam filters, etc

Security – Internal Threats

- Once intruder gains access to a computer, he can trick the memory manager into allowing a process to access main memory cells outside its allotted area.
- This enables him replacing the OS from the memory with his own malicious code.
- Modern CPUs have special-purpose registers in which OS can store the upper and lower limits of process's allotted memory area.
- Everytime a process requests access to a memory cell, CPU checks if the address is within that range.
- If the requested memory address is outside process's memory range, CPU is taken over by OS.
- In order to provide access to those special-purpose registers, CPU has to support 2 modes:
 - **Privilege mode:** held by OS kernel only, executes all machine language instructions specified by CPU
 - **Nonprivilege mode:** held by other processes, can execute only limited machine instructions

Suggested Reading/Activity

- To Read:
 - Command line basics, <http://lifehacker.com/5633909/who-needs-a-mouse-learn-to-use-the-command-line-for-almost-anything>
- To Learn:
 - Codecademy, Command Line, <https://www.codecademy.com/learn/learn-the-command-line>
 - Udacity Course on Linux Command Line, <https://www.udacity.com/course/linux-command-line-basics--ud595>
 - Operating Systems, <https://youtu.be/6toMlfZVc9c>
 - Memory Management, <https://youtu.be/b0T-FeuSipM>
- To Watch:
 - How PCs and Macs differ, <https://youtu.be/SZXY5990xZs>
 - How Linux is Built, <https://youtu.be/yVpbFMhOAwE>
 - Password Cracking, <https://youtu.be/7U-RbOKanYs>
 - What is Bootstrapping? https://youtu.be/nsly1s0U9_c